PLAYWRIGHT AUTOMATION TESTING CAPSTONE PROJECT REPORT (WIPRO)

: - By Rahul Ranjan Singh

TABLE OF CONTENT

1- PROJECT TITLE

End-to-End Test Automation Framework using Playwright for Automation Exercise Website

1.1 WEBSITE NAME

AUTOMATION EXERCISE

1.2 WEBSITE LINK

https://automationexercise.com/

1.3 TECHNOLOGY USED

Automation Tool: Playwright

Programming Language: JavaScript

Reporting Tool: Allure Report

Version Control: Git & GitHub

2- PROJECT OBJECTIVES

This project focuses on developing a comprehensive End-to-End (E2E) Test Automation Framework using Playwright for the Automation Exercise website. The framework automates critical business workflows and validates the functionality of various modules available on the website.

The automation suite covers multiple user journeys, including authentication, product search, cart management, checkout process, shipping validation, homepage verification, and API testing.

The framework follows modern automation testing practices such as:

- Page Object Model (POM)
- Reusable utilities
- Data-driven testing
- API and UI testing
- Allure reporting
- GitHub integration
- CI/CD readiness

3- GOAL

3.1 Functional Goals

- Verify website functionality automatically.
- Reduce manual testing effort.
- Improve test execution speed.
- Detect defects early in development.
- Ensure application stability after changes.

3.2 Technical Goals

- Build a scalable Playwright framework.
- Implement Page Object Model architecture.
- Generate detailed test execution reports.
- Integrate source control using GitHub.
- Support future CI/CD pipeline implementation.

4- SERVICES TESTING

1- **AUTHENTICATION SERVICES: -** Signup and Login
2- **HOME SERVICES: -** Home Page
3- **SEARCH SERVICES: -** Search page and Add to Cart Page
4- **CART SERVICES: -** Cart Page Management
5- **CHECKOUT SERVICES: -** Checkout Page
6- **SHIPPING SERVICES: -** Shipping and Address Page
7- **API SERVICES: -** APIS

5- ANALYSIS AND PLANNING

REQUIREMENT ANALYSIS

The website functionalities were analysed and divided into seven major services:

1- Authentication Module

Features:

1- User Signup
2- User Login
3- User Logout

2- Home Page Module

Features:

- Home Page Loading
- Banner Verification
- Navigation Menu Validation
- Featured Products Display

3- Search Page Module

Features:

- Product Search
- Search Result Validation
- Empty Search Handling

4- Cart Management Module

Features:

- Add Product to Cart
- Remove Product
- Update Quantity
- Cart Verification

5- Checkout Module

Features:

- Checkout Navigation
- Order Summary Verification
- Payment Flow Validation

6- Checkout Module

Features:

- Address Validation
- Shipping Information Verification
- Delivery Details Confirmation

7- API Testing Module

Features:

- Get Products List
- Get Brands List
- Verify Status Codes
- Validate Response Data

TEST STRATEGY

Testing Types Implemented

- Functional Testing
- UI Testing
- API Testing
- Smoke Testing
- Regression Testing
- End-to-End Testing

6- TESTCASE INFORMATION

6.1 AUTHENTICATION TEST CASES

Test Case ID	Description	Priority
SIGNUP-01	Verify user can successfully register with valid data	High
SIGNUP-02	Verify mandatory field validation during user registration	High
LOGIN-01	Verify user can log in successfully with valid credentials	High
LOGIN-02	Verify error message is displayed for invalid password	High
LOGIN-03	Verify validation message appears when login credentials are empty	High
LOGIN-04	Verify email field validation for invalid email format	Medium
LOGIN-05	Verify login email input field is visible on the login page	Low
LOGIN-06	Verify login password input field is visible on the login page	Low
LOGOUT-01	Verify logout button is visible after successful login	Medium
LOGOUT-02	Verify user can successfully log out and is redirected appropriately	High

6.2 HOME PAGE TEST CASE

Test Case ID	Description	Priority
HOME-01	Verify home page loads successfully and all key elements are displayed correctly	High
HOME-02	Verify Logout button is displayed and navigates user to the logout process correctly	High
HOME-03	Verify Delete Account button is displayed and performs account deletion successfully	High
HOME-04	Verify Home button is displayed and redirects user to the home page when clicked	Medium
HOME-05	Verify API Testing button is displayed and navigates to the API Testing page correctly	Medium
HOME-06	Verify Products button is displayed and redirects user to the Products page correctly	High
HOME-07	Verify Cart button is displayed and redirects user to the Cart page correctly	High
HOME-08	Verify Test Cases button is displayed and navigates to the Test Cases page correctly	Medium
HOME-09	Verify Video Tutorials button is displayed and redirects user to the Video Tutorials page correctly	Low
HOME-10	Verify Contact Us button is displayed and navigates to the Contact Us page correctly	Medium
HOME-11	Verify website logo is displayed and redirects to the Home page when clicked	Medium
HOME-12	Verify left carousel arrow is displayed and navigates to the previous banner/slide correctly	Low
HOME-13	Verify right carousel arrow is displayed and navigates to the next banner/slide correctly	Low
HOME-14	Verify category section is displayed and category filters function correctly	Medium
HOME-15	Verify featured items section is displayed and featured products are accessible	High
HOME-16	Verify brands section is displayed and brand filters function correctly	Medium

6.3 SEARCH PAGE TEST CASE

Test Case ID	Description	Priority
SEARCH-01	Verify Products page opens successfully and displays product listings	High
SEARCH-02	Verify Search input field is displayed and accepts user input	High
SEARCH-03	Verify Search button is displayed and triggers product search functionality	High
SEARCH-04	Verify products can be searched successfully using valid keywords	High
SEARCH-05	Verify product images are displayed correctly in search results	Medium
SEARCH-06	Verify product prices are displayed correctly in search results	High
SEARCH-07	Verify product names/descriptions are displayed correctly in search results	High
SEARCH-08	Verify Add to Cart button is displayed and functional for products	High
SEARCH-09	Verify Brands section is displayed and brand filtering functionality works correctly	Medium
SEARCH-10	Verify Women category is displayed and navigates to the appropriate product listing	Medium
SEARCH-11	Verify Kids category is displayed and navigates to the appropriate product listing	Medium
SEARCH-12	Verify Men category is displayed and navigates to the appropriate product listing	Medium
SEARCH-13	Verify first product is displayed properly in the product listing section	Medium
SEARCH-14	Verify Product Details page opens successfully when a product is selected	High
SEARCH-15	Verify product can be added to the cart successfully from the Products page	High
SEARCH-16	Verify Continue Shopping option is displayed and functions correctly after adding a product to the cart	Medium
SEARCH-17	Verify View Cart option is displayed and redirects to the Cart page correctly after adding a product	High

6.4 CART MANAGMENT TEST CASE

Test Case ID	Description	Priority
CART-01	Verify Products page opens successfully and displays product listings	High
CART-02	Verify Cart icon is displayed and accessible to the user	High
CART-03	Verify Products button is displayed and accessible to the user	Medium
CART-04	Verify Cart page opens successfully when Cart button is clicked	High
CART-05	Verify Cart page content loads successfully and is visible to the user	High
CART-06	Verify Cart page displays the correct cart-related information and text	High
CART-07	Verify navigation from Cart module to Products page works correctly	Medium
CART-08	Verify Cart button redirects user to the Cart page correctly	High
CART-09	Verify page title contains expected application name and branding	Low
CART-10	Verify current URL contains the expected Automation Exercise domain	Low
CART-11	Verify Cart page URL is correct after navigation	Medium
CART-12	Verify Products page URL is correct after navigation	Medium
CART-13	Verify Cart button is enabled and clickable	Medium
CART-14	Verify Products button is enabled and clickable	Medium
CART-15	Verify page loads successfully without errors or broken elements	High

6.5 CHECKOUT TEST CASE

Test Case ID	Description	Priority
CHECKOUT-01	Verify Products button is visible and accessible to the user	Medium
CHECKOUT-02	Verify Cart page opens successfully when navigating from the checkout workflow	High
CHECKOUT-03	Verify footer section is visible and displayed correctly	Low
CHECKOUT-04	Verify Products page opens successfully from the checkout workflow	Medium
CHECKOUT-05	Verify Contact Us page navigation works correctly	Medium
CHECKOUT-06	Verify Cart button is enabled and clickable	High
CHECKOUT-07	Verify Subscription section and text are displayed correctly	Low
CHECKOUT-08	Verify Home page navigation works correctly	Medium
CHECKOUT-09	Verify Products button is enabled and clickable	Medium
CHECKOUT-10	Verify Cart page contains the expected cart information and text	High
CHECKOUT-11	Verify page title contains the expected application branding	Low
CHECKOUT-12	Verify Products page content is loaded and visible successfully	Medium
CHECKOUT-13	Verify current URL contains the Automation Exercise domain	Low
CHECKOUT-14	Verify Home button is visible and accessible	Medium
CHECKOUT-15	Verify Contact Us button is visible and accessible	Medium
CHECKOUT-16	Verify Logout button is visible and accessible after successful login	High

6.6 SHIPPING TEST CASE

Test Case ID	Description	Priority
SHIPPING-01	Verify Cart page opens successfully during the shipping workflow	High
SHIPPING-02	Verify Products page opens successfully from the shipping workflow	Medium
SHIPPING-03	Verify Cart page content loads successfully and is visible	High
SHIPPING-04	Verify Delivery Address section is available and can be displayed during checkout	High
SHIPPING-05	Verify Billing Address section is available and can be displayed during checkout	High
SHIPPING-06	Verify Comment Box is available for entering order-related notes	Medium
SHIPPING-07	Verify Place Order button is available for order submission	High
SHIPPING-08	Verify Review Order section is available before placing the order	High
SHIPPING-09	Verify page body loads successfully and is visible	Medium
SHIPPING-10	Verify navigation flow between Products page and Cart page works correctly	High
SHIPPING-11	Verify Products button is visible and accessible	Medium
SHIPPING-12	Verify Cart button is visible and accessible	Medium
SHIPPING-13	Verify Products button is enabled and clickable	Medium
SHIPPING-14	Verify Cart button is enabled and clickable	Medium
SHIPPING-15	Verify Products page content is loaded and visible successfully	Medium
SHIPPING-16	Verify Cart page URL contains the expected "view_cart" path	Medium
SHIPPING-17	Verify current URL contains the Automation Exercise domain	Low
SHIPPING-18	Verify page title contains the expected application branding	Low
SHIPPING-19	Verify footer section is visible and displayed correctly	Low
SHIPPING-20	Verify Subscription section is visible and accessible to the user	Low

6.7 API TEST CASE

Test Case ID	Description	Priority
API-01	Verify API response contains the **title** field for a single post	High
API-02	Verify GET Posts API returns a successful response (Status Code 200)	High
API-03	Verify response headers are returned successfully	Medium
API-04	Verify retrieved post contains the correct Post ID	High
API-05	Verify API response object is returned and is not empty	High
API-06	Verify new post can be created successfully using POST API	High
API-07	Verify API response contains the **userId** field	High
API-08	Verify GET Single Post API returns a successful response	High
API-09	Verify response body contains valid post content	High
API-10	Verify GET Posts API returns one or more records	Medium
API-11	Verify existing post can be updated successfully using PUT API	High
API-12	Verify API response time is within acceptable limits (less than 3 seconds)	Medium
API-13	Verify User ID exists in the API response	High
API-14	Verify response title contains valid data	High
API-15	Verify API returns Status Code 200 for a successful request	High
API-16	Verify response Content-Type is application/json	High
API-17	Verify API returns data in array format when expected	Medium
API-18	Verify API response contains the **body** field	High
API-19	Verify post can be deleted successfully using DELETE API	High
API-20	Verify API response is returned in valid JSON format	High

7- FILE STRUCTURE

```
|-----.github
|-----allure report
|----- allure result
|-----node modules
|-----playwright report
|-----POM
        |-----cart.js
        |-----login.js
        |-----signup.js
        |-----search.js
        |-----shipping.js
        |-----home.js
        |-----checkout.js
|-----test result
|-----tests
        |-----API
                |-----api.spec.js
        |-----Authentication
                        |-----authentication.spec.js
        |-----cart management
                         |-----cart.spec.js
        |-----checkout
                  |----checkout.spec.js
        |-----home
                  |-----home.spec.js
        |-----search
                  |-----search.spec.js
        |-----shipping
                  |-----shipping.spec.js
|-----.env
|-----.gitignore
|-----package-lock.json
|-----package.json
|-----playwright.config.js
```

8- CODE

8.1 POM FILE

8.1.1 Cart.js

```
class CartPage {

  constructor(page) {

    this.page = page;

    // Navigation
    this.productsBtn =
      page.locator("a[href='/products']").first();

    this.cartBtn =
      page.locator("a[href='/view_cart']").first();

  }

  async goToProductsPage() {

    await this.productsBtn.click();

  }

  async openCart() {

    await this.cartBtn.click();

  }

}

module.exports = { CartPage };
```

8.1.2 checkout.js

```
class CheckoutPage {
  constructor(page) {
    this.page = page;

    // Navigation
    this.productsBtn = page.locator("a[href='/products']").first();
    this.cartBtn = page.locator("a[href='/view_cart']").first();
    this.homeBtn = page.locator("a[href='/']").first();
    this.contactBtn = page.locator("a[href='/contact_us']").first();
    this.logoutBtn = page.locator("a[href='/logout']").first();
```

```
    // Common Elements
    this.body = page.locator('body');
    this.footer = page.locator('footer');
    this.subscriptionText = page.locator('body');
  }

  async goToProductsPage() {
    await this.productsBtn.click();
  }

  async openCart() {
    await this.cartBtn.click();
  }

  async openHomePage() {
    await this.homeBtn.click();
  }

  async openContactPage() {
    await this.contactBtn.click();
  }

  async openLoginPage() {
    await this.loginBtn.click();
  }
}

module.exports = { CheckoutPage };
```

8.1.3 home.js

```
class HomePage {

    constructor(page) {
        this.page = page;
        this.productsBtn = page.locator("a[href='/products']").first();
        this.cartBtn = page.locator("a[href='/view_cart']").first();
        this.homeBtn = page.locator("a[href='/']").first();
        this.contactBtn = page.locator("a[href='/contact_us']").first();
        this.loginBtn = page.locator("a[href='/login']").first();
        this.logoutBtn = page.locator("a[href='/logout']").first();
        this.deleteaccountBtn = page.locator("a[href='/delete_account']").first();
        this.testcasesBtn = page.locator("a[href='/test_cases']").first();
        this.apiBtn = page.locator("a[href='/api_list']").first();
        this.videoBtn =
page.locator("a[href='https://www.youtube.com/c/AutomationExercise']").first();
        this.logo = page.locator('.logo.pull-left');
        this.nextSlideBtn = page.locator('[data-slide="next"]').first();
        this.prevSlideBtn = page.locator('[data-slide="prev"]').first();
    }
```

```
}

module.exports = { HomePage };
```

8.1.4 login.js

```
class LoginPage {

    constructor(page) {
        this.page = page;

        // Locators
        this.email = page.locator('input[data-qa="login-email"]');
        this.password = page.locator('input[data-qa="login-password"]');
        this.loginBtn = page.locator('button[data-qa="login-button"]');

        this.logoutBtn = page.locator('a[href="/logout"]');
    }

    // Navigate to login page
    async gotoLoginPage() {
        await this.page.goto('https://automationexercise.com/login');
    }

    // Login action
    async login(email, password) {
        await this.email.fill(email);
        await this.password.fill(password);
        await this.loginBtn.click();
    }

    // Logout action
    async logout() {
        await this.logoutBtn.click();
    }
}

module.exports = LoginPage;
```

8.1.5 search.js

```
class SearchPage {

    constructor(page) {

        this.page = page;
        this.searchInput =page.locator('#search_product');
        this.searchBtn =page.locator('#submit_search');
        this.addToCartBtn =page.locator('.add-to-cart').first();
        this.productDetailsBtn =page.locator('a[href*="/product_details/"]').first();
```

```
    }

    async gotoProductsPage() {

        await this.page.goto('https://automationexercise.com/products',{waitUntil:
'domcontentloaded',timeout: 60000});

    }

}

module.exports = { SearchPage };
```

8.1.6 shipping.js

```
class ShippingPage {
  constructor(page) {
    this.page = page;

    // Navigation
    this.cartBtn = page.locator("a[href='/view_cart']").first();
    this.productsBtn = page.locator("a[href='/products']").first();

    // Shipping / Checkout Elements
    this.checkoutBtn = page.locator('a:has-text("Proceed To Checkout")');
    this.deliveryAddress = page.locator('#address_delivery');
    this.billingAddress = page.locator('#address_invoice');
    this.commentBox = page.locator('textarea[name="message"]');
    this.placeOrderBtn = page.locator('a:has-text("Place Order")');
    this.reviewOrderSection = page.locator('#cart_info');

    // Common Elements
    this.body = page.locator('body');
    this.footer = page.locator('footer');
    this.subscriptionText = page.locator('body');
  }

  async openProductsPage() {
    await this.productsBtn.click();
  }

  async openCartPage() {
    await this.cartBtn.click();
  }

  async proceedToCheckout() {
    await this.checkoutBtn.click();
  }

  async enterComment(message) {
    await this.commentBox.fill(message);
  }
```

```
  async placeOrder() {
    await this.placeOrderBtn.click();
  }
}

module.exports = { ShippingPage };
```

8.1.7 signup.js

```
class SignupPage {

    constructor(page) {
        this.page = page;

        this.name = page.locator('input[data-qa="signup-name"]');
        this.email = page.locator('input[data-qa="signup-email"]');
        this.signupBtn = page.locator('button[data-qa="signup-button"]');
    }

    async gotoSignupPage() {
        await this.page.goto('https://automationexercise.com/login');
    }

    async signup(name, email) {
        await this.name.fill(name);
        await this.email.fill(email);
        await this.signupBtn.click();
    }
}

module.exports = SignupPage;
```

8.2 TESTS FILE

8.2.1 Authentication

|----- 8.2.1.1 signup.spec.js

```
// @ts-check
const { test, expect } = require('@playwright/test');
const SignupPage = require('../../POM/signup.js');

test.describe('Signup Tests', () => {

    // ================================
    // AUTH-01 Signup with valid data
    // ================================

    test('AUTH-01 Signup with valid data', async ({ page }) => {

        const signupPage = new SignupPage(page);

        await signupPage.gotoSignupPage();

        await signupPage.signup('Rahul',`rahul${Date.now()}@gmail.com`);

        await expect(page.locator('text=Enter Account Information')).toBeVisible();

    });

    // ================================
    // AUTH-02 Mandatory fields validation
    // ================================

        test('AUTH-02 Mandatory fields validation', async ({ page }) => {
        const signupPage = new SignupPage(page);
        await signupPage.gotoSignupPage();
        await signupPage.signup('Rahul',`rahul${Date.now()}@gmail.com`);
        await expect(page.locator('text=Enter Account Information')).toBeVisible();
        await page.locator('button[data-qa="create-account"]').click();
        await expect(page.locator('text=Enter Account Information')).toBeVisible();

    });

});
```

8.2.1.2 Login.spec.js

```
const { test, expect } = require('@playwright/test');
const LoginPage = require('../../POM/login.js');
require('dotenv').config();

test.describe('Login Tests', () => {
    // ================================
```

```
    // LOGIN-01 valid Login
    // ================================

    test('LOGIN-01 valid Login', async ({ page }) => {
        const loginPage = new LoginPage(page);
        await loginPage.gotoLoginPage();
        await loginPage.login(process.env.EMAIL, process.env.PASSWORD);

        await page.waitForTimeout(3000);

        await expect(page.locator('text=Logged in as')).toBeVisible();

    });
    // ================================
    // LOGIN-02 invalid password
    // ================================

    test('LOGIN-02 invalid password', async ({ page }) => {

        const loginPage = new LoginPage(page);
        await loginPage.gotoLoginPage();
        await loginPage.login(process.env.EMAIL, 'wrongpassword');
        await expect(page.locator('text=Your email or password is
incorrect!')).toBeVisible();

    });

    // ================================
    // LOGIN-03 empty credentials
    // ================================

    test('LOGIN-03 empty credentials', async ({ page }) => {

        const loginPage = new LoginPage(page);
        await loginPage.gotoLoginPage();
        await loginPage.login('', '');
        await expect(page.locator('input[data-qa="login-email"]')).toBeVisible();

    });


    // ================================
    // LOGIN-04 invalid email format
    // ================================
    const invalidEmails = [
        'rahulgmail.com',
        'rahul@gmail',
        '@gmail.com',
        'rahul@.com',
        'rahul.com',
        'rahul@@gmail.com',
        'rahul gmail@gmail.com',
        'rahul#gmail.com'];
```

```
    invalidEmails.forEach(email => {
        test('LOGIN-04 invalid email format' + email, async ({ page }) => {

            const loginPage = new LoginPage(page);
            await loginPage.gotoLoginPage();
            await page.waitForTimeout(3000);
            await loginPage.login(email, process.env.PASSWORD);

            await expect(page).toHaveURL('https://automationexercise.com/login', {
timeout: 10000 });
        });
    });
    // ================================
    // LOGIN-05 login email field visible
    // ================================

    test('LOGIN-05 login email field visible', async ({ page }) => {

        const loginPage = new LoginPage(page);

        await loginPage.gotoLoginPage();

        await expect(page.locator('input[data-qa="login-email"]')).toBeVisible();

    });

    // ================================
    // LOGIN-06 login password field visible
    // ================================

    test('LOGIN-06 login password field visible', async ({ page }) => {

        const loginPage = new LoginPage(page);

        await loginPage.gotoLoginPage();

        await expect(page.locator('input[data-qa="login-password"]')).toBeVisible();

    });
    // ================================
    // LOGIN-07 verify logout button visible
    // ================================

    test('LOGIN-07 verify logout button visible', async ({ page }) => {

        const loginPage = new LoginPage(page);
        await loginPage.gotoLoginPage();
        await loginPage.login(process.env.EMAIL, process.env.PASSWORD);
        await expect(page.locator('a[href="/logout"]')).toBeVisible();

    });

    // ================================
    // LOGIN-08 verify Sucessfull logout
```

```
    // ================================

    test('LOGIN-08 verify Sucessfull logout', async ({ page }) => {

        const loginPage = new LoginPage(page);
        await loginPage.gotoLoginPage();
        await loginPage.login(process.env.EMAIL, process.env.PASSWORD);
        await loginPage.logout();
        await expect(page).toHaveURL('https://automationexercise.com/login');

    });
});
```

8.2.2 api.spec.js

```
const { test, expect } = require('@playwright/test');

test.describe('API Testing Module', () => {
  const baseUrl = 'https://jsonplaceholder.typicode.com';

  // ================================
  // API-01 Validate Post Contains Title
  // ================================
  test('API-01 Validate Post Contains Title', async ({ request }) => {
    const response = await request.get(`${baseUrl}/posts/1`);
    const body = await response.json();
    expect(body).toHaveProperty('title');
  });

  // ================================
  // API-02 GET Posts API
  // ================================
  test('API-02 GET Posts API', async ({ request }) => {
    const response = await request.get(`${baseUrl}/posts`);
    expect(response.status()).toBe(200);
  });

  // ================================
  // API-03 Validate Response Header Exists
  // ================================
  test('API-03 Validate Response Header Exists', async ({ request }) => {
    const response = await request.get(`${baseUrl}/posts`);
    expect(response.headers()).toBeTruthy();
  });

  // ================================
  // API-04 Validate Post ID
  // ================================
  test('API-04 Validate Post ID', async ({ request }) => {
    const response = await request.get(`${baseUrl}/posts/1`);
    const body = await response.json();
    expect(body.id).toBe(1);
  });
```

```
// ================================
// API-05 Validate API Response Object
// ================================
test('API-05 Validate API Response Object', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  const body = await response.json();
  expect(body).toBeTruthy();
});

// ================================
// API-06 POST Create Post API
// ================================
test('API-06 POST Create Post API', async ({ request }) => {
  const response = await request.post(`${baseUrl}/posts`, {
    data: { title: 'Playwright', body: 'API Testing', userId: 1 }
  });
  expect(response.status()).toBe(201);
});

// ================================
// API-07 Validate Post Contains UserId
// ================================
test('API-07 Validate Post Contains UserId', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  const body = await response.json();
  expect(body).toHaveProperty('userId');
});

// ================================
// API-08 GET Single Post API
// ================================
test('API-08 GET Single Post API', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  expect(response.ok()).toBeTruthy();
});

// ================================
// API-09 Validate Response Body
// ================================
test('API-09 Validate Response Body', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  const body = await response.json();
  expect(body.body).toBeTruthy();
});

// ================================
// API-10 Validate Total Posts Returned
// ================================
test('API-10 Validate Total Posts Returned', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts`);
  const body = await response.json();
  expect(body.length).toBeGreaterThan(0);
});
```

```
// ================================
// API-11 PUT Update Post API
// ================================
test('API-11 PUT Update Post API', async ({ request }) => {
  const response = await request.put(`${baseUrl}/posts/1`, {
    data: { id: 1, title: 'Updated Title', body: 'Updated Body', userId: 1 }
  });
  expect(response.ok()).toBeTruthy();
});

// ================================
// API-12 Validate API Response Time
// ================================
test('API-12 Validate API Response Time', async ({ request }) => {
  const start = Date.now();
  await request.get(`${baseUrl}/posts`);
  const end = Date.now();
  expect(end - start).toBeLessThan(3000);
});

// ================================
// API-13 Validate User ID Exists
// ================================
test('API-13 Validate User ID Exists', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  const body = await response.json();
  expect(body.userId).toBeTruthy();
});

// ================================
// API-14 Validate Response Title
// ================================
test('API-14 Validate Response Title', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  const body = await response.json();
  expect(body.title).toBeTruthy();
});

// ================================
// API-15 Validate Status Code 200
// ================================
test('API-15 Validate Status Code 200', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  expect(response.status()).toBe(200);
});

// ================================
// API-16 Validate Content Type
// ================================
test('API-16 Validate Content Type', async ({ request }) => {
  const response = await request.get(`${baseUrl}/posts/1`);
  expect(response.headers()['content-type']).toContain('application/json');
});
```

```
  // ================================
  // API-17 Validate Array Response
  // ================================
  test('API-17 Validate Array Response', async ({ request }) => {
    const response = await request.get(`${baseUrl}/posts`);
    const body = await response.json();
    expect(Array.isArray(body)).toBeTruthy();
  });

  // ================================
  // API-18 Validate Post Contains Body
  // ================================
  test('API-18 Validate Post Contains Body', async ({ request }) => {
    const response = await request.get(`${baseUrl}/posts/1`);
    const body = await response.json();
    expect(body).toHaveProperty('body');
  });

  // ================================
  // API-19 DELETE Post API
  // ================================
  test('API-19 DELETE Post API', async ({ request }) => {
    const response = await request.delete(`${baseUrl}/posts/1`);
    expect(response.status()).toBe(200);
  });

  // ================================
  // API-20 Validate Response Is JSON
  // ================================
  test('API-20 Validate Response Is JSON', async ({ request }) => {
    const response = await request.get(`${baseUrl}/posts/1`);
    expect(response.headers()['content-type']).toContain('application/json');
  });
});
```

8.2.3 Cart.spec.js

```
const { test, expect } = require('@playwright/test');
const  LoginPage  = require('../../POM/login.js');
const  {SearchPage}  = require('../../POM/search.js');
const { CartPage } = require('../../POM/cart.js');
require('dotenv').config();

test.describe('Cart Module Tests', () => {
  let loginPage;
  let cartPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
```

```
  cartPage = new CartPage(page);

  // Perform login first
  await loginPage.gotoLoginPage();
  await loginPage.login(process.env.EMAIL, process.env.PASSWORD);

  // Navigate to products page after login
  await cartPage.goToProductsPage();
});

// ================================
// CART-01 Verify products page opens
// ================================
test('CART-01 Verify products page opens', async ({ page }) => {
  await expect(page).toHaveURL(/products/);
});

// ================================
// CART-02 Verify cart icon visible
// ================================
test('CART-02 Verify cart icon visible', async () => {
  await expect(cartPage.cartBtn).toBeVisible();
});

// ================================
// CART-03 Verify products button visible
// ================================
test('CART-03 Verify products button visible', async () => {
  await expect(cartPage.productsBtn).toBeVisible();
});

// ================================
// CART-04 Verify cart page opens
// ================================
test('CART-04 Verify cart page opens', async ({ page }) => {
  await cartPage.openCart();
  await expect(page).toHaveURL(/view_cart/);
});

// ================================
// CART-05 Verify body visible on cart page
// ================================
test('CART-05 Verify body visible on cart page', async ({ page }) => {
  await cartPage.openCart();
  await expect(page.locator('body')).toBeVisible();
});

// ================================
// CART-06 Verify page contains cart text
// ================================
test('CART-06 Verify page contains cart text', async ({ page }) => {
  await cartPage.openCart();
  await expect(page.locator('body')).toContainText('Cart');
});
```

```
// ================================
// CART-07 Verify product page navigation
// ================================
test('CART-07 Verify product page navigation', async ({ page }) => {
  await cartPage.goToProductsPage();
  await expect(page).toHaveURL(/products/);
});

// ================================
// CART-08 Verify cart button navigation
// ================================
test('CART-08 Verify cart button navigation', async ({ page }) => {
  await cartPage.openCart();
  await expect(page).toHaveURL(/view_cart/);
});

// ================================
// CART-09 Verify page title contains automation
// ================================
test('CART-09 Verify page title contains automation', async ({ page }) => {
  await expect(page).toHaveTitle(/Automation/);
});

// ================================
// CART-10 Verify current url contains automation exercise
// ================================
test('CART-10 Verify current url contains automation exercise', async ({ page }) => {
  expect(page.url()).toContain('automationexercise');
});

// ================================
// CART-11 Verify cart page url
// ================================
test('CART-11 Verify cart page url', async ({ page }) => {
  await cartPage.openCart();
  expect(page.url()).toContain('view_cart');
});

// ================================
// CART-12 Verify products url
// ================================
test('CART-12 Verify products url', async ({ page }) => {
  await cartPage.goToProductsPage();
  expect(page.url()).toContain('products');
});

// ================================
// CART-13 Verify cart button enabled
// ================================
test('CART-13 Verify cart button enabled', async () => {
  await expect(cartPage.cartBtn).toBeEnabled();
});
```

```
  // ================================
  // CART-14 Verify products button enabled
  // ================================
  test('CART-14 Verify products button enabled', async () => {
    await expect(cartPage.productsBtn).toBeEnabled();
  });

  // ================================
  // CART-15 Verify page loaded successfully
  // ================================
  test('CART-15 Verify page loaded successfully', async ({ page }) => {
    await expect(page.locator('body')).toBeVisible();
  });

});
```

8.2.4 Checkout.spec.js

```
const { test, expect } = require('@playwright/test');
const LoginPage = require('../../POM/login.js');
const { CartPage } = require('../../POM/cart.js');
const { SearchPage } = require('../../POM/search.js');
const { CheckoutPage } = require('../../POM/checkout.js');
require('dotenv').config();

test.describe('Checkout And Order Module', () => {
    let loginPage, cartPage, searchPage, checkoutPage;

    test.beforeEach(async ({ page }) => {
        loginPage = new LoginPage(page);
        cartPage = new CartPage(page);
        searchPage = new SearchPage(page);
        checkoutPage = new CheckoutPage(page);

        // Login first
        await loginPage.gotoLoginPage();
        await loginPage.login(process.env.EMAIL, process.env.PASSWORD);
    });

    // ================================
    // CHECKOUT-01 Verify products button visible
    // ================================
    test('CHECKOUT-01 Verify products button visible', async () => {
        await expect(checkoutPage.productsBtn).toBeVisible();
    });

    // ================================
    // CHECKOUT-02 Verify cart page opens
    // ================================
    test('CHECKOUT-02 Verify cart page opens', async ({ page }) => {
        await checkoutPage.openCart();
```

```
    await expect(page).toHaveURL(/view_cart/);
});

// ================================
// CHECKOUT-03 Verify footer visible
// ================================
test('CHECKOUT-03 Verify footer visible', async () => {
    await expect(checkoutPage.footer).toBeVisible();
});

// ================================
// CHECKOUT-04 Verify products page opens
// ================================
test('CHECKOUT-04 Verify products page opens', async ({ page }) => {
    await checkoutPage.goToProductsPage();
    await expect(page).toHaveURL(/products/);
});

// ================================
// CHECKOUT-05 Verify contact page navigation
// ================================
test('CHECKOUT-05 Verify contact page navigation', async ({ page }) => {
    await checkoutPage.openContactPage();
    await expect(page.url()).toContain('contact_us');
});

// ================================
// CHECKOUT-06 Verify cart button enabled
// ================================
test('CHECKOUT-06 Verify cart button enabled', async () => {
    await expect(checkoutPage.cartBtn).toBeEnabled();
});

// ================================
// CHECKOUT-07 Verify subscription text available
// ================================
test('CHECKOUT-07 Verify subscription text available', async () => {
    await expect(checkoutPage.subscriptionText).toContainText('Subscription');
});

// ================================
// CHECKOUT-08 Verify home page navigation
// ================================
test('CHECKOUT-08 Verify home page navigation', async ({ page }) => {
    await checkoutPage.openHomePage();
    await expect(page.url()).toContain('automationexercise');
});

// ================================
// CHECKOUT-09 Verify products button enabled
// ================================
test('CHECKOUT-09 Verify products button enabled', async () => {
    await expect(checkoutPage.productsBtn).toBeEnabled();
});
```

```
    // ================================
    // CHECKOUT-10 Verify cart page contains cart text
    // ================================
    test('CHECKOUT-10 Verify cart page contains cart text', async () => {
        await checkoutPage.openCart();
        await expect(checkoutPage.body).toContainText('Cart');
    });

    // ================================
    // CHECKOUT-11 Verify page title contains automation
    // ================================
    test('CHECKOUT-11 Verify page title contains automation', async ({ page }) => {
        await expect(page).toHaveTitle(/Automation/);
    });

    // ================================
    // CHECKOUT-12 Verify body visible on products page
    // ================================
    test('CHECKOUT-12 Verify body visible on products page', async () => {
        await checkoutPage.goToProductsPage();
        await expect(checkoutPage.body).toBeVisible();
    });

    // ================================
    // CHECKOUT-13 Verify current url contains automationexercise
    // ================================
    test('CHECKOUT-13 Verify current url contains automationexercise', async ({ page }) =>
{
        expect(page.url()).toContain('automationexercise');
    });

    // ================================
    // CHECKOUT-14 Verify home button visible
    // ================================
    test('CHECKOUT-14 Verify home button visible', async () => {
        await expect(checkoutPage.homeBtn).toBeVisible();
    });

    // ================================
    // CHECKOUT-15 Verify contact button visible
    // ================================
    test('CHECKOUT-15 Verify contact button visible', async () => {
        await expect(checkoutPage.contactBtn).toBeVisible();
    });

    // ================================
    // CHECKOUT-16 Verify logout button visible
    // ================================
    test('CHECKOUT-16 Verify logout button visible', async ({page}) => {
        await expect(checkoutPage.logoutBtn).toBeVisible();
    });

});
```

8.2.5 home.spec.js

```
const { test, expect } = require('@playwright/test');
const LoginPage = require('../../POM/login.js');
const { HomePage } = require('../../POM/home.js');
require('dotenv').config();

test.describe('Home Page Tests', () => {
    let loginPage;
    let homePage;

    test.beforeEach(async ({ page }) => {
        loginPage = new LoginPage(page);
        homePage = new HomePage(page);

        // Login first
        await loginPage.gotoLoginPage();
        await loginPage.login(process.env.EMAIL, process.env.PASSWORD);
    });

    // ================================
    // HOME-01 Verify home page visible successfully
    // ================================
    test('HOME-01 Verify home page visible successfully', async ({ page }) => {
        await expect(page.locator('body')).toContainText('Home');
     });

    // ================================
    // HOME-02 Verify logout button visible
    // ================================
    test('HOME-02 Verify logout button visible', async () => {
        await expect(homePage.logoutBtn).toBeVisible();
     });

    // ================================
    // HOME-03 Verify delete account button visible
    // ================================
    test('HOME-03 Verify delete account button visible', async () => {
        await expect(homePage.deleteaccountBtn).toBeVisible();
     });

    // ================================
    // HOME-04 Verify home page button visible
    // ================================
    test('HOME-04 Verify home page button visible', async () => {
        await expect(homePage.homeBtn).toBeVisible();
     });

    // ================================
    // HOME-05 Verify API list button visible
    // ================================
    test('HOME-05 Verify API list button visible', async () => {
        await expect(homePage.apiBtn).toBeVisible();
     });
```

```
// ================================
// HOME-06 Verify Product button visible
// ================================
test('HOME-06 Verify Product button visible', async () => {
    await expect(homePage.productsBtn).toBeVisible();
 });

// ================================
// HOME-07 Verify cart button visible
// ================================
test('HOME-07 Verify cart button visible', async () => {
    await expect(homePage.cartBtn).toBeVisible();
 });

// ================================
// HOME-08 Verify Test Cases button visible
// ================================
test('HOME-08 Verify Test Cases button visible', async () => {
    await expect(homePage.testcasesBtn).toBeVisible();
 });

// ================================
// HOME-09 Verify Video Tutorials button visible
// ================================
test('HOME-09 Verify Video Tutorials button visible', async () => {
    await expect(homePage.videoBtn).toBeVisible();
 });

// ================================
// HOME-10 Verify Contact Us button visible
// ================================
test('HOME-10 Verify Contact Us button visible', async () => {
    await expect(homePage.contactBtn).toBeVisible();
 });

// ================================
// HOME-11 Verify Logo visible
// ================================
test('HOME-11 Verify Logo visible', async () => {
    await expect(homePage.logo).toBeVisible();
});

// ================================
// HOME-12 Verify Left Arrow visible
// ================================
test('HOME-12 Verify Left Arrow visible', async () => {
    await expect(homePage.prevSlideBtn).toBeVisible();
});

// ================================
// HOME-13 Verify Right Arrow visible
// ================================
test('HOME-13 Verify Right Arrow visible', async () => {
```

```
        await expect(homePage.nextSlideBtn).toBeVisible();
    });

    // ================================
    // HOME-14 Verify Category visible
    // ================================
    test('HOME-14 Verify Category visible', async ({ page }) => {
        await expect(page.locator('.left-sidebar')).toContainText(/Category/i);
    });

    // ================================
    // HOME-15 Verify Feature Items visible
    // ================================
    test('HOME-15 Verify Feature Items visible', async ({page}) => {
        await expect(page.locator('.title.text-center').first()).toContainText(/Features
Items/i);
    });

    // ================================
    // HOME-16 Verify Brand visible
    // ================================
    test('HOME-16 Verify Brand visible', async ({page}) => {
        await expect(page.locator('.brands_products')).toContainText(/Brands/i);
    });
});
```

8.2.6 Search.spec.js

```
const { test, expect } = require('@playwright/test');
const LoginPage = require('../../POM/login.js');
const { SearchPage } = require('../../POM/search.js');
require('dotenv').config();

test.describe('Product Module Tests', () => {
    let loginPage;
    let productPage;

    test.beforeEach(async ({ page }) => {
        loginPage = new LoginPage(page);
        productPage = new SearchPage(page);
        await loginPage.gotoLoginPage();
        await loginPage.login(process.env.EMAIL, process.env.PASSWORD);
        await productPage.gotoProductsPage();

    });

    // ====================================
    // SEARCH-01 Verify Products Page Opens
    // ====================================
    test('SEARCH-01 Verify Products Page Opens', async ({ page }) => {
```

```
    await expect(page.locator('body')).toContainText('All Products');

});

// ====================================
// SEARCH-02 Verify Search Input Visible
// ====================================
test('SEARCH-02 Verify Search Input Visible', async ({ page }) => {

    await expect(page.locator('#search_product')).toBeVisible();

});

// ====================================
// SEARCH-03 Verify Search Button Visible
// ====================================
test('SEARCH-03 Verify Search Button Visible', async ({ page }) => {

    await expect(page.locator('#submit_search')).toBeVisible();

});

// ====================================
// SEARCH-04 Verify Search Product
// ====================================

test(`SEARCH-04 Verify Search Product`, async ({ page }) => {

    await page.fill('#search_product', 'Top');
    await page.click('#submit_search');
    await expect(page.locator('body')).toContainText('Top');

});

// ====================================
// SEARCH-05 Verify Image VIsible
// ====================================
test('SEARCH-05 Verify Image VIsible', async ({ page }) => {
    await page.fill('#search_product', 'Top');
    await page.click('#submit_search');
    await expect(page.locator('.product-image-wrapper').first()).toBeVisible();

});

// ====================================
// SEARCH-06 Verify Price VIsible
// ====================================

test('SEARCH-06 Verify Price VIsible', async ({ page }) => {
    await page.fill('#search_product', 'Top');
    await page.click('#submit_search');
    await expect(page.locator('.product-image-wrapper').first()).toContainText('500');

});
```

```
    // ====================================
    // SEARCH-07 Verify Text VIsible
    // ====================================
    test('SEARCH-07 Verify Text VIsible', async ({ page }) => {
        await page.fill('#search_product', 'Top');
        await page.click('#submit_search');
        await expect(page.locator('.product-image-wrapper').first()).toContainText('Blue
Top');

    });

    // ====================================
    // SEARCH-08 Verify Add to Cart VIsible
    // ====================================
    test('SEARCH-08 Verify Add to Cart VIsible', async ({ page }) => {
        await page.fill('#search_product', 'Top');
        await page.click('#submit_search');
        await expect(page.locator('[data-product-id="1"]').first()).toContainText('Add to
cart');

    });

    // ====================================
    // SEARCH-09 Verify Brands Section Visible
    // ====================================
    test('SEARCH-09 Verify Brands Section Visible', async ({ page }) => {

        await expect(page.locator('body')).toContainText('Brands');

    });

    // ====================================
    // SEARCH-10 Verify Women Category Visible
    // ====================================
    test('SEARCH-10 Verify Women Category Visible', async ({ page }) => {

        await expect(page.locator('body')).toContainText('Women');

    });

    // ====================================
    // SEARCH-11 Verify Kids Category Visible
    // ====================================
    test('SEARCH-11 Verify Kids Category Visible', async ({ page }) => {

        await expect(page.locator('body')).toContainText('Kids');

    });

    // ====================================
    // SEARCH-12 Verify Men Category Visible
    // ====================================
    test('SEARCH-12 Verify Men Category Visible', async ({ page }) => {
```

```
    await expect(page.locator('body')).toContainText('Men');

});


// ====================================
// SEARCH-13 Verify first product visible
// ====================================
test('SEARCH-13 Verify first product visible', async ({ page }) => {

    await expect(page.locator('.product-image-wrapper').first()).toBeVisible();

});

// ====================================
// SEARCH-14 Verify product details page opens
// ====================================
test('SEARCH-14 Verify product details page opens', async ({ page }) => {

    await page.locator('a[href*="/product_details/"]').first().click({ force: true });

    await expect(page.locator('body')).toContainText('Availability');

});

// ====================================
// SEARCH-15 Verify Add To Cart
// ====================================
    test('SEARCH-15 Verify Add To Cart', async ({ page }) => {

    await page.locator('.add-to-cart').first().click({ force: true });

    await expect(page.locator('body')).toContainText('Added!');

});

// ============================================
// SEARCH-16 Verify Continue Shopping Visible
// ============================================
test('SEARCH-16 Verify Continue Shopping Visible', async ({ page }) => {

    await page.locator('.add-to-cart').first().click({ force: true });

    await expect(page.locator('.modal-content')).toContainText('Continue Shopping');

});

// ============================================
// SEARCH-17 Verify View Cart Visible
// ============================================
test('SEARCH-17 Verify View Cart Visible', async ({ page }) => {

    await page.locator('.add-to-cart').first().click({ force: true });
```

```
        await expect(page.locator('.modal-content')).toContainText('View Cart');

    });



});
```

8.2.7 Shipping.spec.js

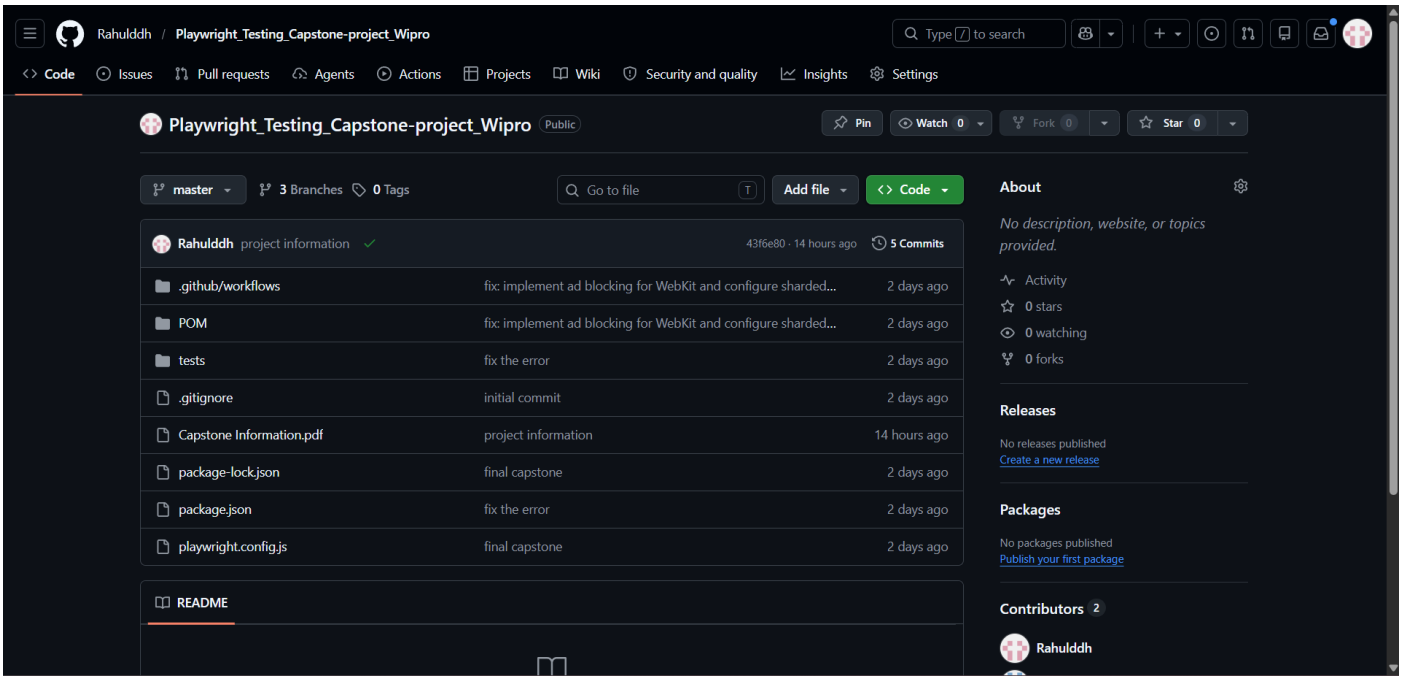
```
const { test, expect } = require('@playwright/test');
const LoginPage = require('../../POM/login.js');
const { CartPage } = require('../../POM/cart.js');
const { SearchPage } = require('../../POM/search.js');
const { ShippingPage } = require('../../POM/shipping.js');
require('dotenv').config();

test.describe('Shipping Address Module', () => {
  let loginPage, cartPage, searchPage, shippingPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    cartPage = new CartPage(page);
    searchPage = new SearchPage(page);
    shippingPage = new ShippingPage(page);

    // Login first
    await loginPage.gotoLoginPage();
    await loginPage.login(process.env.EMAIL, process.env.PASSWORD);
  });

  // ================================
  // SHIPPING-01 Verify cart page opens
  // ================================
  test('SHIPPING-01 Verify cart page opens', async ({ page }) => {
    await shippingPage.openCartPage();
    await expect(page.url()).toContain('view_cart');
  });

  // SHIPPING-02 Verify products page opens
  test('SHIPPING-02 Verify products page opens', async ({ page }) => {
    await shippingPage.openProductsPage();
    await expect(page.url()).toContain('products');
  });

  // SHIPPING-03 Verify cart page body visible
  test('SHIPPING-03 Verify cart page body visible', async () => {
    await shippingPage.openCartPage();
    await expect(shippingPage.body).toBeVisible();
  });

  // SHIPPING-04 Verify delivery address section locator created
  test('SHIPPING-04 Verify delivery address section locator created', async () => {
    await expect(shippingPage.deliveryAddress).toBeHidden();
```

```
});

// SHIPPING-05 Verify billing address section locator created
test('SHIPPING-05 Verify billing address section locator created', async () => {
  await expect(shippingPage.billingAddress).toBeHidden();
});

// SHIPPING-06 Verify comment box locator created
test('SHIPPING-06 Verify comment box locator created', async () => {
  await expect(shippingPage.commentBox).toBeHidden();
});

// SHIPPING-07 Verify place order button locator created
test('SHIPPING-07 Verify place order button locator created', async () => {
  await expect(shippingPage.placeOrderBtn).toBeHidden();
});

// SHIPPING-08 Verify review order section locator created
test('SHIPPING-08 Verify review order section locator created', async () => {
  await expect(shippingPage.reviewOrderSection).toBeHidden();
});

// SHIPPING-09 Verify body visible
test('SHIPPING-09 Verify body visible', async () => {
  await expect(shippingPage.body).toBeVisible();
});

// SHIPPING-10 Verify shipping flow navigation
test('SHIPPING-10 Verify shipping flow navigation', async ({ page }) => {
  await shippingPage.openProductsPage();
  await shippingPage.openCartPage();
  await expect(page.url()).toContain('view_cart');
});

// SHIPPING-11 Verify products button visible
test('SHIPPING-11 Verify products button visible', async () => {
  await expect(shippingPage.productsBtn).toBeVisible();
});

// SHIPPING-12 Verify cart button visible
test('SHIPPING-12 Verify cart button visible', async () => {
  await expect(shippingPage.cartBtn).toBeVisible();
});

// SHIPPING-13 Verify products button enabled
test('SHIPPING-13 Verify products button enabled', async () => {
  await expect(shippingPage.productsBtn).toBeEnabled();
});

// SHIPPING-14 Verify cart button enabled
test('SHIPPING-14 Verify cart button enabled', async () => {
  await expect(shippingPage.cartBtn).toBeEnabled();
});
```

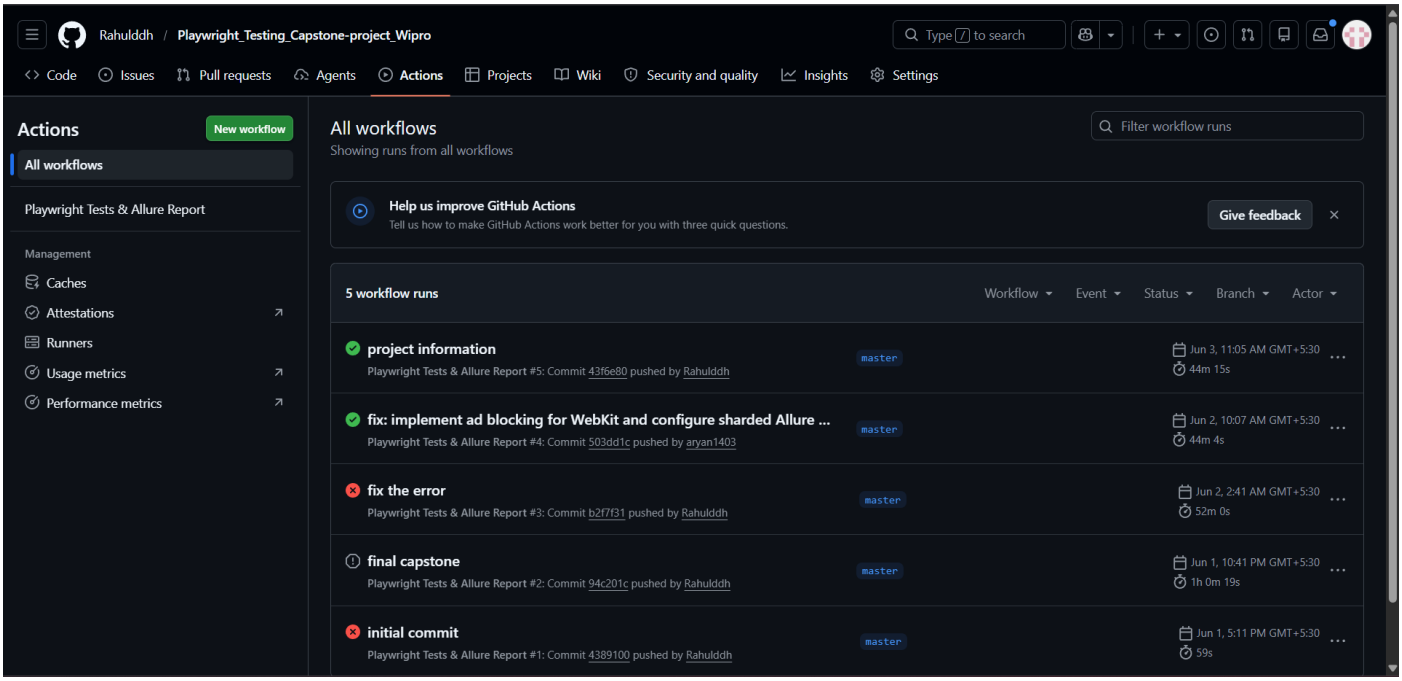
```
  // SHIPPING-15 Verify products page body visible
  test('SHIPPING-15 Verify products page body visible', async () => {
    await shippingPage.openProductsPage();
    await expect(shippingPage.body).toBeVisible();
  });

  // SHIPPING-16 Verify cart page url contains view cart
  test('SHIPPING-16 Verify cart page url contains view cart', async ({ page }) => {
    await shippingPage.openCartPage();
    await expect(page.url()).toContain('view_cart');
  });

  // SHIPPING-17 Verify current url contains automationexercise
  test('SHIPPING-17 Verify current url contains automationexercise', async ({ page }) => {
    expect(page.url()).toContain('automationexercise');
  });

  // SHIPPING-18 Verify page title contains automation
  test('SHIPPING-18 Verify page title contains automation', async ({ page }) => {
    await expect(page).toHaveTitle(/Automation/);
  });

  // SHIPPING-19 Verify footer visible
  test('SHIPPING-19 Verify footer visible', async () => {
    await expect(shippingPage.footer).toBeVisible();
  });

  // SHIPPING-20 Verify subscription section visible
  test('SHIPPING-20 Verify subscription section visible', async () => {
    await expect(shippingPage.subscriptionText).toContainText('Subscription');
  });
});
```

9- GITHUB

9.1 GITHUB REPOSITORY STRUCTURE

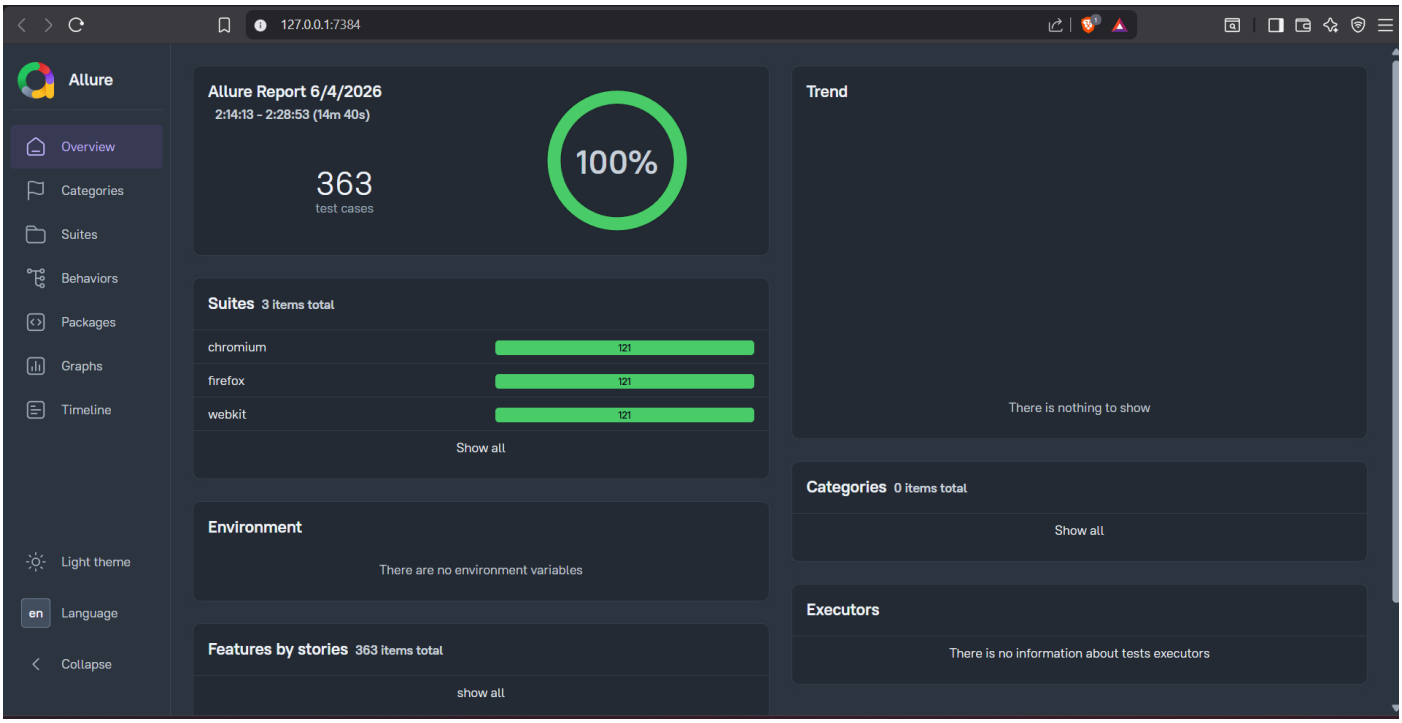


9.2 GITHUB ACTION WORKFLOW

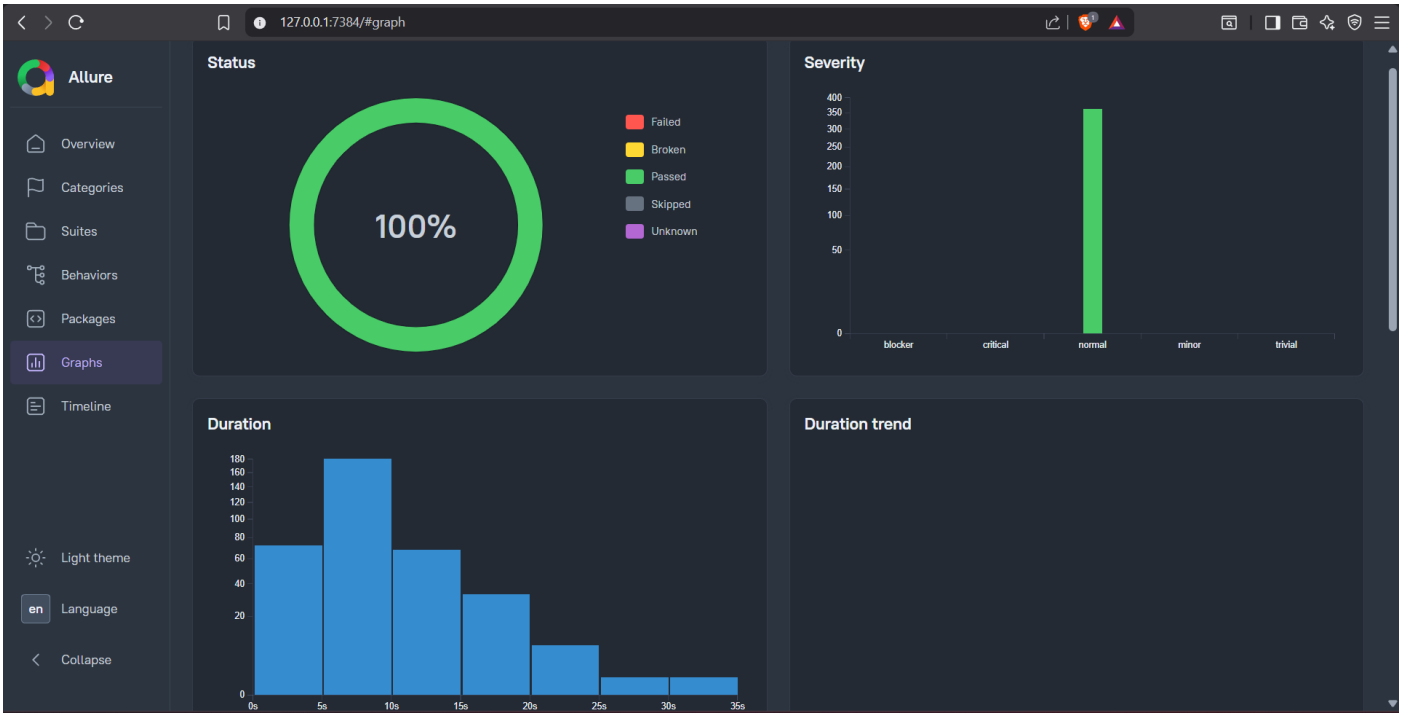


10- ALLURE REPORT

10.1 ALLURE DASHBOARD OVERVIEW



10.2 ALLURE GRAPH AND ANALYTICS



11- CONCULUSION

This Playwright Automation Testing Project successfully automated the major functional areas of the Automation Exercise website.
The framework covers:

- Authentication Testing
- Homepage Validation
- Product Search Testing
- Cart Management Testing
- Checkout Process Testing
- Shipping Validation Testing
- API Testing

Key achievements include:

- Page Object Model implementation.
- Reusable and maintainable test architecture.
- Automated UI and API validation.
- Comprehensive Allure reporting.
- GitHub version control integration.
- Scalable structure for future enhancements.

The project significantly reduces manual testing effort, improves test reliability, and provides fast feedback regarding application quality. The framework can be easily integrated into CI/CD pipelines for continuous testing and deployment processes.

Overall, this project demonstrates practical implementation of modern test automation techniques using Playwright and serves as a production-ready automation framework for web application testing.